

COS30002 AI for Games

Task 18 – The Custom Project Plan

AN OVERVIEW OF A TRACK B PRACTICAL TANKS-LIKE GAME AI SYSTEM OUTCOME

Written by Edward Herrod

104541862

Contents

Section 1 – The Core Concept and Alignment with Unit Learning Outcomes	2
Potential extension features:	4
Section 2 – The Technical Stack and Architecture.....	4
Section 3 – The Artifact Blueprint	5
Physical implementation	5
Documentation	5
Demo presentation	5

Section 1 – The Core Concept and Alignment with Unit Learning Outcomes

The conceptual framework around which I will be working comes from the 2006 game *Wii Play* for the Nintendo Wii, most specifically its *Tanks!* minigame where the player controls a tank on a mission to destroy all enemy tanks and try to make it to as high a level as they can.



Figure 1: A screenshot from *Wii Play's Tanks!*

My custom project will be handled similarly, but will look much simpler due to time, technology and budget constraints, swapping out the isometric view and wooden art style with a top-down view and basic geometry to represent important game assets.

Agents themselves (sans the player agent, who uses their human brain) will have their overarching behaviour controlled by layered finite state machines (FSMs) [A] and navigate the world by utilising the A* pathfinding algorithm [G].

More specifically, each type of enemy tank will get its own set of behaviours [X] that it will follow:

- Brown tanks are dummies – they have no movement logic and shoot bullets sporadically in a pseudo-random direction.
- Grey tanks are grunts – they move and shoot bullets sporadically in pseudo-random directions.
- Teal tanks are basic rocketeers, maintaining distance from the player if possible and shooting rockets towards the player.
- Red tanks are aggressors and will rush the player on sight, barraging them with bullets as they do so.
- Yellow tanks are the front-liners. They can lay mines and will try to take space but maintain a set distance (though smaller than the Teal tanks') from the player whilst doing so.

- Green tanks are snipers – stationary, they will use more complex trajectory calculations, including off walls, to get a rocket shot on any player, even ones they’ve noticed have gone behind cover (they also blend in with the background because it’s green in this game!).
- Purple tanks are defenders, identifying hiding/cover spots and staying near them to force the player to their location.
- **Black tanks are elites and will use cover as leverage to peek corners and try to survive without getting overly ambitious. They’re fast, attempting to dodge the players’ bullets, and fire rockets directly at them. They can also lay mines. Scary stuff.**

Agent movement will not be momentum based but is instead stop-and-start, with subtle acceleration but no inertia or momentum. This is to place more focus on the variable AI types and intelligent pathfinding [G] for some of them.

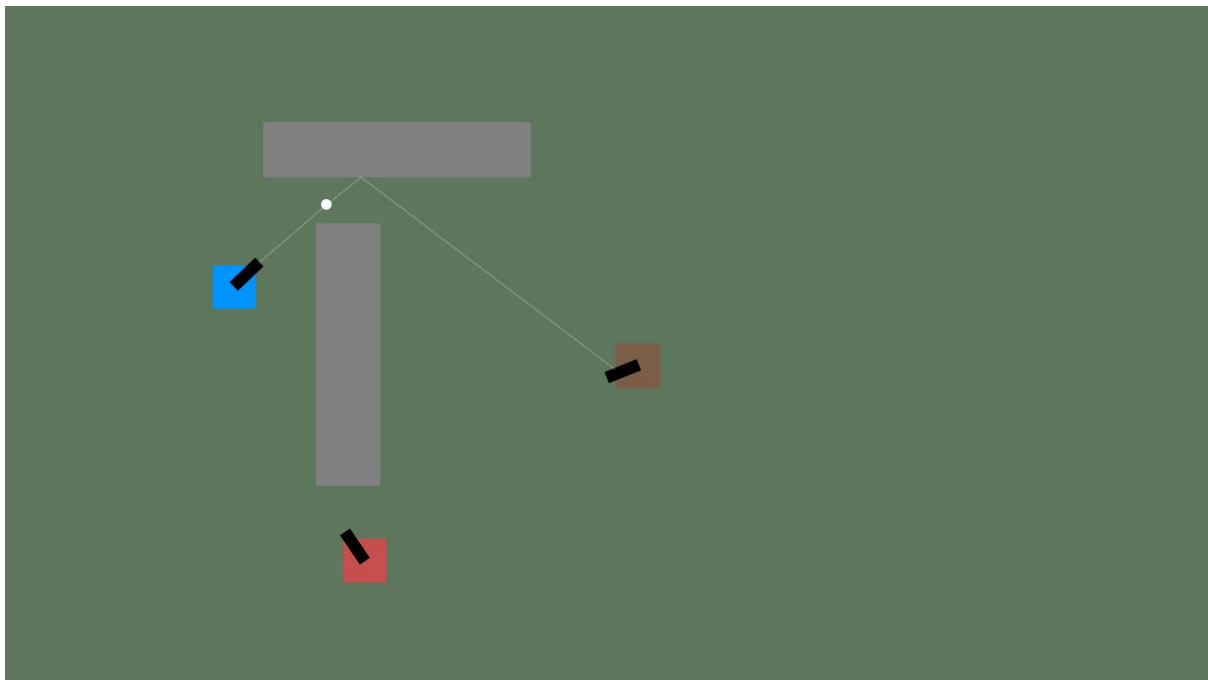


Figure 2: A mock-up of the game world on any given level. Tank agents, including the player, are rendered as square chassis with black swivelling gun barrels on top. The player blue tank agent shoots an unmoving (i.e. `movement_speed = 0`, to potentially facilitate variable movement speeds) “dummy” brown tank agent by ricocheting a round off a wall (the bullet’s projected trajectory is highlighted in white). A red “aggressor” tank agent sneaks up from behind cover to engage the player in combat.

Potential extension features:

To extend the demonstration of my understanding, I could also add the following:

- 100 (like in *Tanks!*), or even Infinite levels, and a score system based on how fast you can clear a stage [A]
- A scary, white, invisible boss tank AI with more than 1 health [X]
- Destructible environments that get blown up by mine explosions and AI planning around this (yellow and black tanks could utilise it) [P]
- Powerups that AI tank agents will seek out and pick up (not the player!) such as one which duplicates an enemy and has the resultant group travel together using a flocking algorithm [S]

Section 2 – The Technical Stack and Architecture

I acknowledge that my custom project will be coded in the Python language as required. I will pair this with the Pyglet game development library, as that is what I have become accustomed to over the course of this unit. I also acknowledge that the use of Heavy engines such as Unity, Unreal or Godot are prohibited to place focus on the development of AI algorithms.

My codebase will be structured with modularity and extensibility in mind, as this program should have the capability to add as many different AI agents and levels as were specified in Section 1, and possibly more (for instance, a “boss” AI). The levels will use text files for their map layouts and agent spawn locations, similarly to the *Task 6 Spike – Navigation with Graphs*. Each different colour of tank agent will get their own logic to determine which states they will be in at any given time, but to avoid code duplication, this will be implemented similarly to the way I did so in the *Task 16 Spike – Soldier on Patrol*, with one fsm.py file guiding the decision-making for every AI tank agent.

Section 3 – The Artifact Blueprint

Physical implementation

I will code the working game loop simulation and maps. I will also provide visual debugging tools which can be activated with a key press, as well as the algorithms that the AI will use to path find and aim.

Documentation

I will document instructions on how to use my code in a README.md file, which will lay out all the controls and features of the simulation.

Demo presentation

I will present a demonstration of my working simulation live. If necessary, I can also provide a YouTube video presentation of my simulation running.